Cognitive Robotics Report

Assessment 1: Shortest-path Navigation using Turtlebot3 in Gazebo and RViz

Elliot Alladayè: ea102@kent.ac.uk

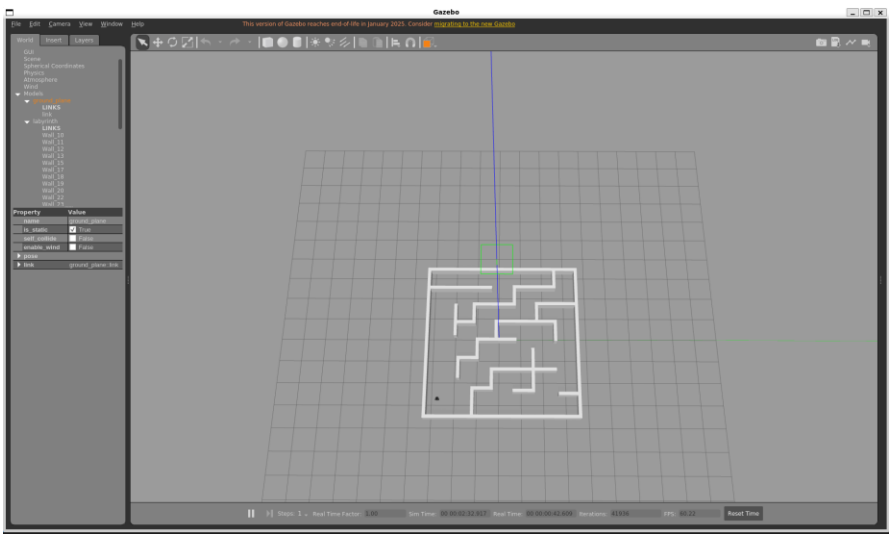
Jean-Baptiste Azan: ja104@kent.ac.uk

Jérémy Bisson: jb110@kent.ac.uk

1. The Navigation Environment

1.1 Create Gazebo World

We created the map using Gazebo's the Building Editor, we used the https://multimazegenerator.com/index.html website to generate a maze that we used as inspiration. We used Gazebo's floor plan option to open the map as a 2D floorplan and scaled it so that it would respect the dimensionality requirements. We saved it as a '.world' file.



1.2 Create a Launch file to load the world and spawn the robot

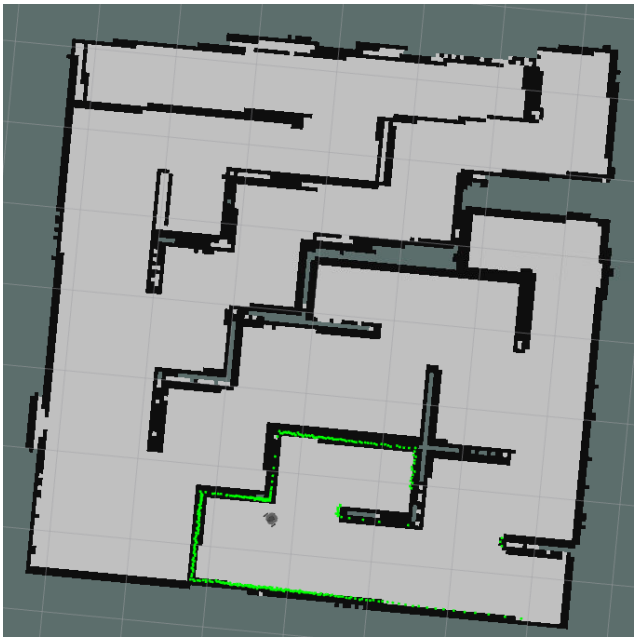
To make sure the robot spawns in the bottom left corner of the map, we modified the x, y, and z coordinates. We also made sure to assign custom_world.world to the world_name argument to launch gazebo.

1.3 SLAM the Gazebo World

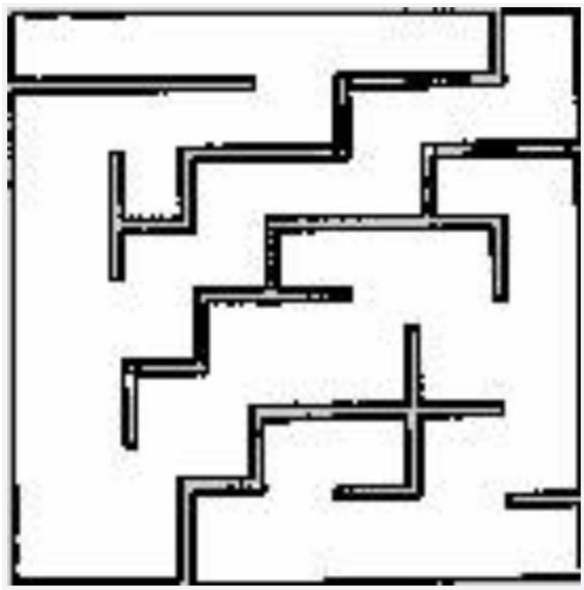
To choose the SLAM method, we first compared the 4 different options:

- Gmapping: The industry standard for basic 2D mapping, reliable but it requires wheel odometry and is CPU-heavy in large areas.
- Hector SLAM: Best for aerial or uneven terrain, it works without wheel odometry but requires high-end, high-frequency LIDAR.
- Karto SLAM: A modern, graph-based 2D approach that excels in large environments due to its superior "loop closure" (correcting the map when returning to a known spot).
- RTAB-Map: The premier choice for 3D mapping; it uses visual data and advanced memory management to handle massive, complex environments

Based on our needs (a flat terrain and a robot that has wheel odometry), we first chose to use Gmapping based slam. At first, the results were decent but at some point, the results became slightly rotated and not in-par with the actual map , likely because Gmapping relies heavily on a Particle Filter, which can suffer from "particle deprivation" during rapid turns or when wheel slippage occurs (see image below).



Based on this result, we decided to try the Karto SLAM method: it is great for 2D terrain and solves Gmapping's problems with the loop closure functionality. The resulting map is much more coherent (see below the PGM file).



To launch the project, we use 3 different launch files (one of them being invoked through another launch file). The steps to launch the project are:

- Go into the dedicated Docker image or WSL with ROS installed or ubuntu with ROS installed

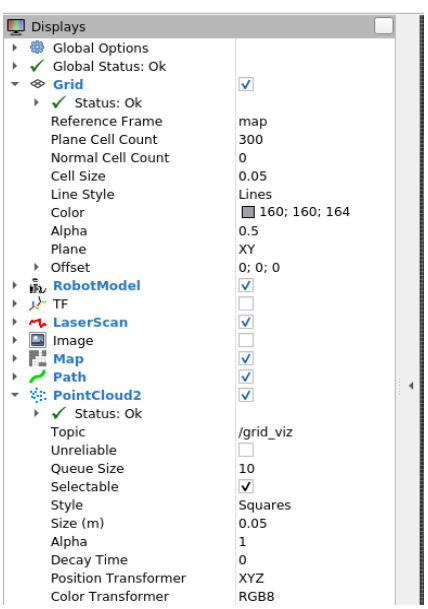
- Open a first terminal and go into the project's root directory
- Execute source devel/setup.bash and roslaunch custom_gazebo_world custom_world.launch
- Open a second terminal and go into the project's root directory
- Execute devel/setup.bash and roslaunch path_planning turtlebot3_custom_world.launch algorithm:=smoothed
- Use "2D Nav Goal" in rviz and click anywhere on the map, it will create a path to the right part of the map by going through 3 intermediate waypoints.

We created a Readme.md file to the project to detail the steps of to launch the project.

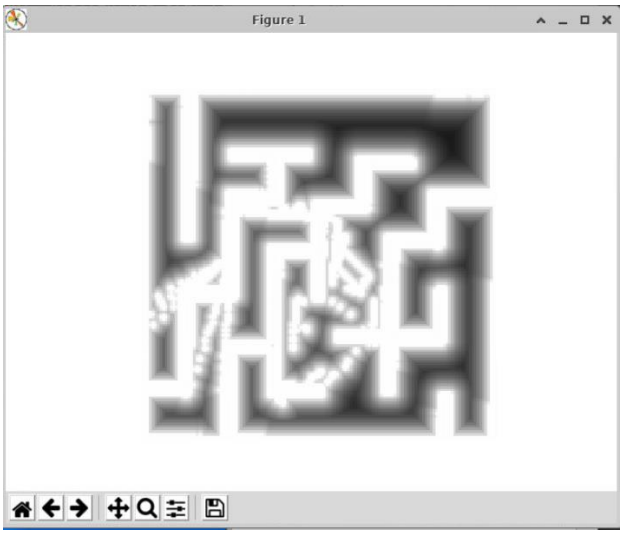
We added the "algorithm" argument to the path_planning launch file; is used to choose between the standard and smoothed A*. It's standard by default.

Once Gazebo and RViz are launched, we normally use the "2D Pose Estimate" tool is used to initialize the AMCL particle filter, providing a starting pose from which the algorithm can begin refining the robot's localized position on the map. We added parameters to the AMCL node for the robot to always have the right coordinates and orientation without any manual input.

We configured some displays in RViz and we saved the configuration file to use it each time we launch the project.



During the project, we faced problems with AMCL and costmap synchronization, to visualize the problem we create the show_maze_with_point function that creates a Matplotlib window with the costmap (see below).



As we can see, having a wrong AMCL initialization creates a distorted costmap. We fixed that problem by changing the map server's frame_id to map instead of odom and by adding default parameters to position the robot

2. Optimised A* algorithm with path smoothing

2.1. Implementation of the algorithm

The A* algorithm is a path finding algorithm that relies on heuristics to find the shortest path to a point. In our implementation (src/path_finding/srcipts/a_star.py), we created a "Cell" class to represent the cells on the costmap. We store the visited cells in a set to ensure O(1) look-up time, preventing the algorithm from re-processing the same nodes.

Core Algorithmic Principles

The implementation follows the fundamental A* cost function:

f(n) = g(n) + h(n)

- g(n) (Path Cost): Represented by `neighbor[1]`, this is the exact cost from the start node to the current cell.
- h(n) (Heuristic): Calculated via `euclidean_dist`. We use Euclidean distance to provide an "admissible" heuristic, meaning it never overestimates the actual cost to the goal, which guarantees that the algorithm finds the optimal path.

Priority Search and "To Visit" List

Instead of a simple breadth-first search, A* prioritizes nodes with the lowest f(n). In our implementation, `find_lowest_f` scans the `to_visit` list (the "Open Set") to select the most promising candidate for expansion.

```
def find_lowest_f(array: list[Cell]):
    # Selects the node with the minimum f-score to explore next
    # This guides the search towards the goal based on the heuristic
```

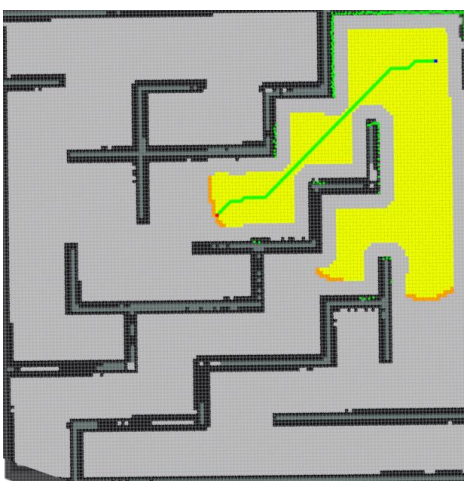
Path Reconstruction

Once the `current.pos == goal` condition is met, the algorithm backtracks using the `parent` reference stored in each `Cell` object. This utilizes the "greedy" property of the explored nodes to reconstruct the optimal sequence of moves from goal to start.

Costmap Integration and Neighbors

The `find_neighbors` function ensures that the agent respects environmental constraints. By passing the `costmap` into the neighbor expansion phase, we prune invalid nodes (obstacles) before they are ever added to the `to_visit` list, ensuring the resulting path is kinematically feasible.

This is the result of the standard A*:



2.2. Integration of the algorithm with the ROS navigation stack

To prioritize time over distance, we modified the cost function to account for the robot's physical limits. By utilizing MAX_LINEAR_VELOCITY and

MAX_ANGULAR_VELOCITY, the algorithm converts spatial movements into temporal costs.

The cost_function calculates the total time for a transition:

```
def cost_function(distance, angle):
    time_trans = distance / MAX_LINEAR_VELOCITY
    time_rot = abs(angle) / MAX_ANGULAR_VELOCITY
    return time_trans + time_rot
```

We determine the rotation requirement by comparing the vector from the parent cell to the current cell against the vector to the potential neighbor. This is implemented using math.atan2 on the cross and dot products of these vectors.

```
ux, uy = previous_x - current_x, previous_y - current_y
vx, vy = neighbor_x - current_x, neighbor_y - current_y
angle = math.atan2(ux * vy - uy * vx, ux * vx + uy * vy)
```

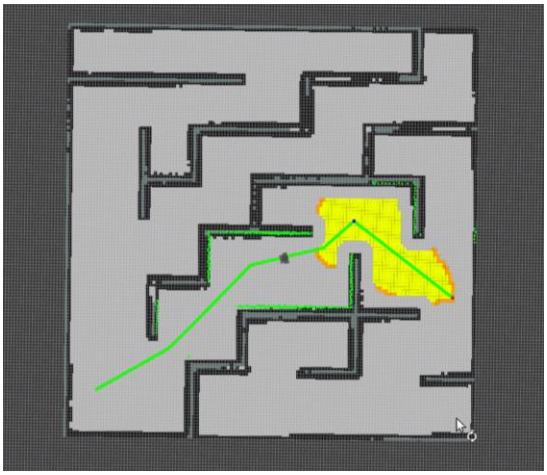
This angular penalty prevents inefficient, jagged paths. The algorithm favors smooth trajectories because frequent turns increase the g-score. To ensure the heuristic remains admissible, we also divide the Euclidean distance to the goal by the maximum linear velocity. This ensures that we never overestimate the time remaining, allowing A* to find the most time-efficient path.

At the end, when the path has been computed, we did some research about line of sight smoothing and we decided to implement the Bresenham's line algorithm which is an algorithm that determines all the cells that need to be traversed to go from a cell A to a cell B.

We can then check if all these cells are collision free and if the path respects the robot footprint, we do this by using a cost threshold. If the path between these two cells is feasible by the robot, we can delete all the cells in the path between these two cells (useless steps).

We iterate over all the cells to delete all the "useless" cells in the path so at the end, the path is only composed of straight lines.

This is the result of the smoothed A*:



As we said earlier, we added the “algorithm” argument to the path_planning launch file. It is used to choose between the standard and smoothed A*. The algorithm choosed is then invoked by the run_algorithm function, inside the make_plan() function.

```
def make_plan(req) -> PathPlanningPluginResponse:
    # This goals are indexes, we got those values by logging req.goal and clicking with 2D
    algo_param = rospy.get_param("~algorithm", "standard")
    available_algorithms = {
        "standard": ("standard", a_star),
        "smoothed": ("smoothed", a_star_smoothed),
    }

    if algo_param in available_algorithms:
        algo_name, algo_func = available_algorithms[algo_param]
    else:
        rospy.logwarn(
            f"Unknown algorithm '{algo_param}', defaulting to standard",
        )
        algo_name, algo_func = available_algorithms["standard"]

    rospy.loginfo(f"Selected algorithm: {algo_name}")

    init_metrics_csv(algo_name)
    rospy.loginfo("=" * 60)
    rospy.loginfo(f"Running algorithm: {algo_name}")
    rospy.loginfo("=" * 60)
    path = run_algorithm(
        algo_func, algo_name, start, goals, width, height, costmap, resolution, origin
    )
```

3. Performance evaluation of the algorithm

3.1. Compare the paths produced by the algorithms and comment on the quality (smoothness, turning behaviour) – optimality trade-off.

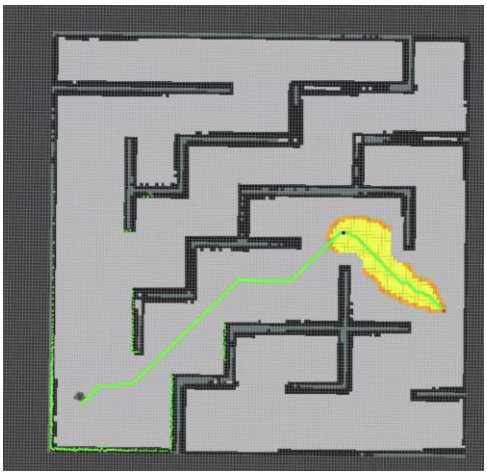
The standard A* algorithm optimizes purely for the shortest distance. Its cost function only considers the traversal cost from the costmap (g = g + neighbor_cost), with a Euclidean distance heuristic. This produces the shortest possible path in terms of grid cells, but the path often contains frequent direction changes and sharp turns, as the algorithm has no incentive to maintain a consistent heading.

The enhanced A* introduces two additional components in the cost function: a turning penalty and a travel-time cost. At each node expansion, the angle between the previous, current, and candidate neighbour node is computed using atan2. This angle is then used in a time-based cost function that accounts for both translation time (distance / MAX_LINEAR_VELOCITY) and rotation time (|angle| / MAX_ANGULAR_VELOCITY), reflecting the physical constraints of the TurtleBot3. The heuristic is also expressed in time units (euclidean_dist / MAX_LINEAR_VELOCITY) to remain consistent and admissible.

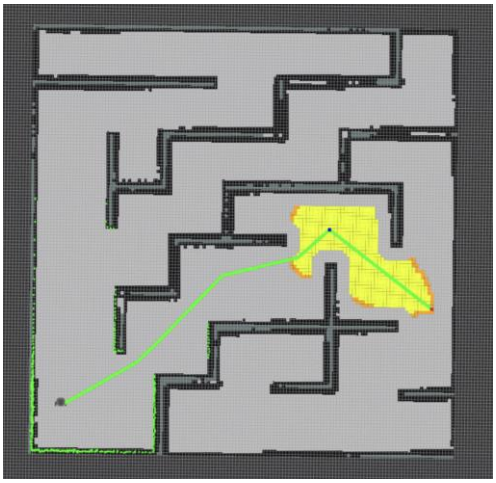
As a result, the enhanced A* produces noticeably smoother paths with fewer and gentler turns. However, these paths tend to be longer in terms of total grid cells, since the algorithm may choose a longer route to avoid sharp direction changes. This is the fundamental trade-off: the standard A* is distance-optimal but produces physically costly

paths (many rotations slow the robot down), while the enhanced A* sacrifices distance optimality to produce paths that are better suited to the robot's kinematic constraints.

Standard path planning:



Smoothed path planning:



3.2 Compare the computation time of the standard A* and the enhanced A* with smoothing. Comment on the impact of turning and travel-time costs on planning time?

The standard A* algorithm consistently computes paths faster than the enhanced A* with smoothing across all four segments. On average, standard A* takes 2.57s per segment versus 4.02s for the smoothed variant, an overhead of approximately 56%.

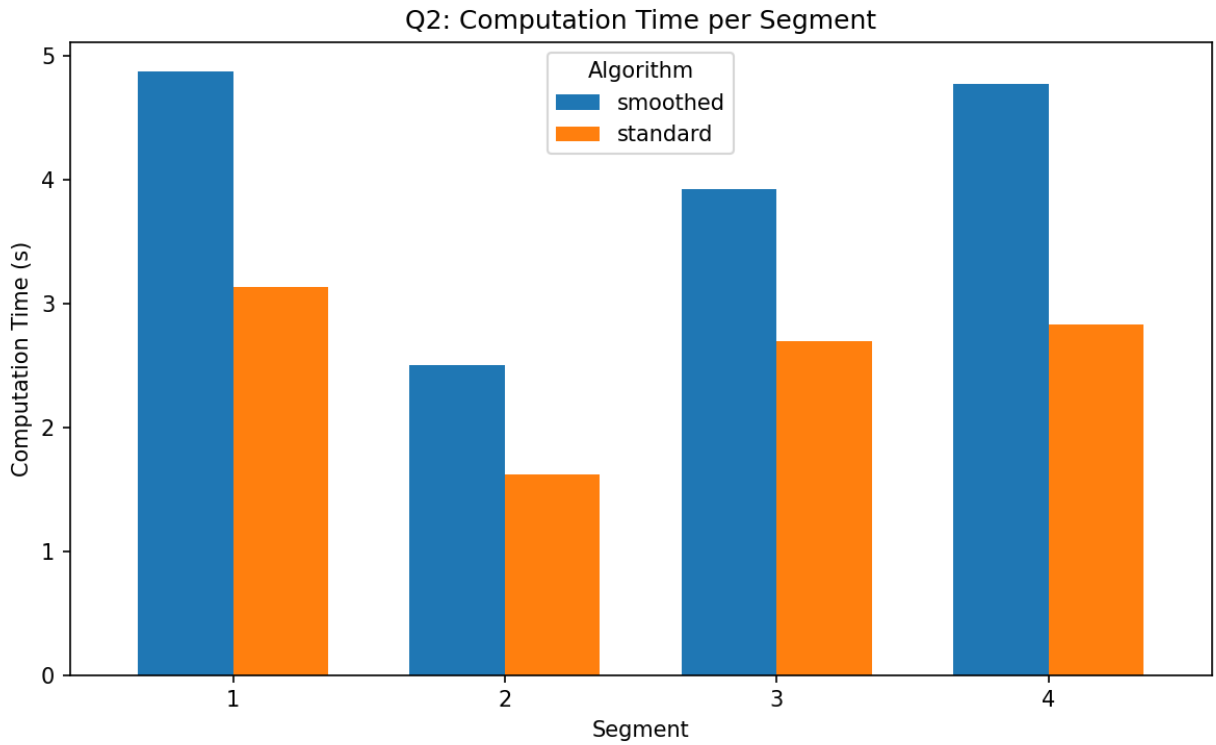
Segment	Standard (seconds)	Smoothed (seconds)	Overhead
1	3.139	4.874	+55%
2	1.620	2.508	+55%
3	2.694	3.926	+46%
4	2.836	4.781	+69%
Average (seconds)	2.572	4.022	+56%

This increased computation time is expected and comes from two sources:

Multi-objective cost function: At each node expansion, the smoothed algorithm computes additional costs, the turning angle penalty (requiring the angle between the previous, current, and candidate nodes) and the travel-time cost (translational + rotational components). These operations are performed for every neighbour evaluation, significantly increasing the per-node cost compared to standard A* which only uses Euclidean distance.

Line-of-sight post-processing (Bresenham): After A* finds the initial path, the algorithm uses Bresenham's line algorithm to check whether intermediate waypoints can be replaced by direct straight-line connections. Each check verifies that no cell along the line exceeds the costmap obstacle threshold. However, this step is computationally lightweight compared to the cost function overhead, as it only iterates over the already-computed path rather than expanding new nodes.

The overhead varies across segments (46%–69%), which can be attributed to the local topology: segment 4, which traverses an area with more junctions, requires more node expansions where the additional cost calculations compound.



3.3 Compare the execution (travel) time for both planners. Based on your specific results, comment on whether and when smoother paths may result in faster execution, followed by observations on path length.

The line-of-sight post-processing dramatically reduces path length: the smoothed algorithm produces paths of only 2-3 waypoints per segment, compared to 22-35 for standard A*. This is the result of Bresenham's line algorithm replacing long sequences of grid-aligned waypoints with direct straight-line connections.

Segment	Standard length	Smoothed length	Standard exec (seconds)	Smoothed exec (seconds)

1	32	3	15.142	12.059 (-20%)
2	22	2	6.993	7.161 (+2%)
3	35	3	10.299	9.683 (-6%)
4	34	2	8.81	8.731 (-1%)

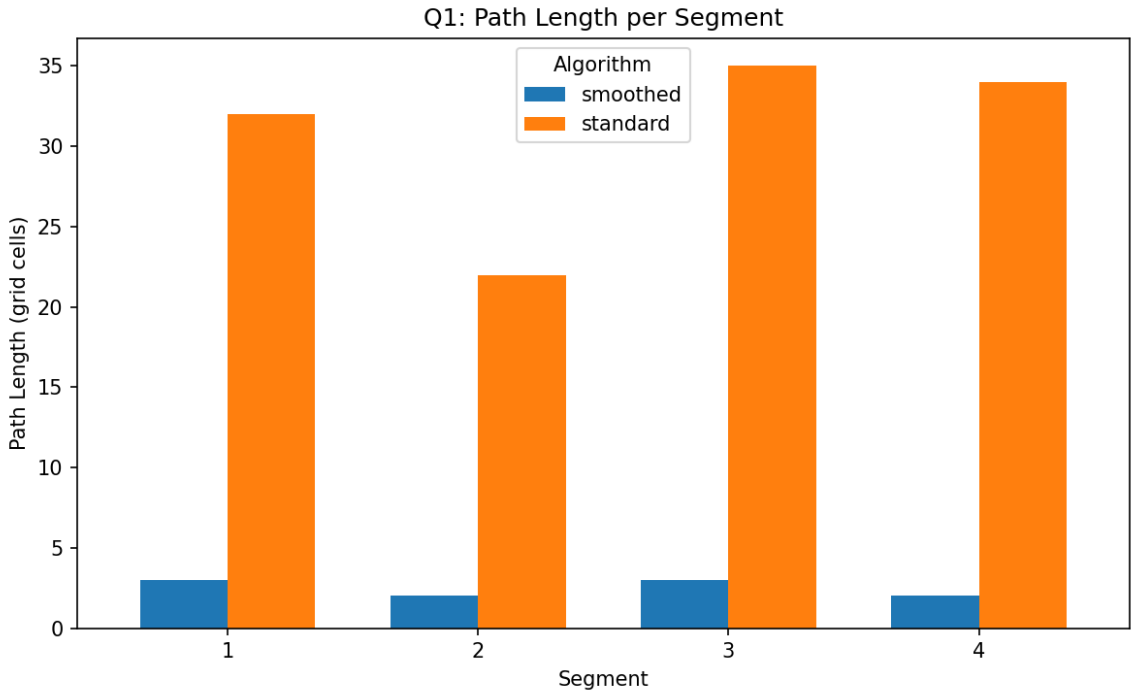
The smoothed algorithm achieves faster execution on 3 out of 4 segments. With fewer waypoints, the robot follows more direct trajectories and spends less time adjusting its heading at intermediate grid-aligned waypoints. The improvement is most significant on segment 1 (-20%), which is the longest segment with the most turns eliminated by the smoothing.

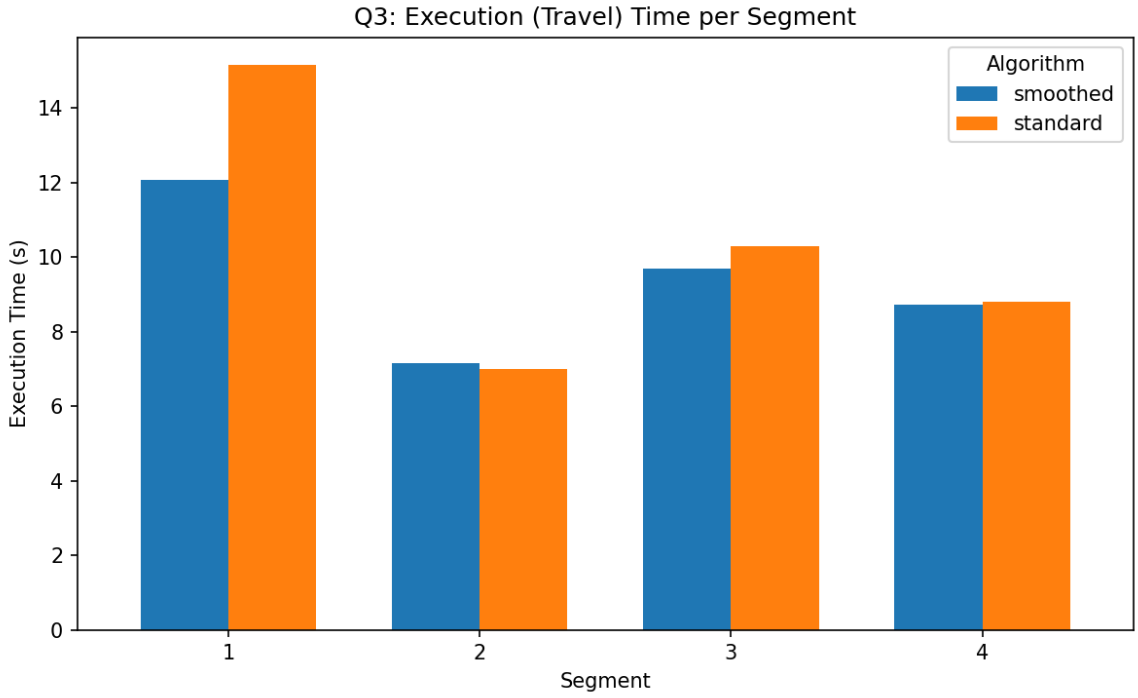
Segment 2 is the exception, where the smoothed algorithm is marginally slower (+2%). This segment is the shortest and most straightforward, meaning the standard path already follows a near-direct route, leaving little room for the smoothing to improve execution.

Despite the large reduction in waypoint count, the execution time improvement remains moderate overall. This is because the DWA local controller already smooths the robot's actual trajectory at runtime, regardless of the global path. The benefit of global path smoothing is therefore most visible on complex segments where the standard path forces DWA to make frequent heading corrections.

To detect goal completion, we compare the current robot's position using the function 'get_robot_position' with the goal position.

```
def get_robot_position():
    """Get current robot position from /odom (blocking, single message)."""
      msg = rospy.wait_for_message("/odom", Odometry, timeout=10.0)
return msg.pose.pose.position.x, msg.pose.pose.position.y
```


Q1: Path Length per Segment
Algorithm
smoothed
standard
Path Length (grid cells)
0
5
10
15
20
25
30
35
1
2
3
4
Segment


Q3: Execution (Travel) Time per Segment
Algorithm
smoothed
standard
Execution Time (s)
0
2
4
6
8
10
12
14
1
2
3
4
Segment

3.4 In questions 1-3, do the observed trends hold across all four navigation segments? Argue why.

Path length: The trend is now strongly consistent, the smoothed algorithm produces dramatically shorter paths on every segment (2-3 waypoints vs 22-35). The line-of-sight post-processing using Bresenham's algorithm eliminates all redundant intermediate waypoints, and this reduction is proportionally similar across all segments.

Computation time: The smoothed algorithm is consistently slower across all four segments, with an overhead ranging from 46% to 69%. The overhead is relatively stable for segments 1-3 (~55%) and increases for segment 4 (69%), which traverses an area with more junctions requiring more node expansions. This trend is robust and predictable.

Execution time: The smoothed algorithm is faster on 3 out of 4 segments, with the most significant improvement on segment 1 (-20%). The exception is segment 2 (+2%), the shortest and most straightforward segment where the standard path already follows a near-direct route. This trend is geometry-dependent: the more complex the segment, the greater the benefit of having fewer, more direct waypoints. We also observed that for both algorithms, the robot was slightly offset from the planned green path in RViz, confirming that the DWA local controller mediates the final trajectory regardless of the global path.

3.5 Explain how line-of-sight post-processing smoothing complements the multi-objective optimised global path. Why, if so, is it beneficial even when the global planner already considers turning and travel-time penalties?

These two techniques are complementary and address different limitations:

The multi-objective cost function operates during search: it biases A* towards paths with fewer sharp turns and lower travel time at the node-expansion level. However, it is still constrained by grid discretisation, the path must follow grid cell edges and can only move in 8 directions. Even an optimally smooth A* path will contain unnecessary intermediate waypoints along straight sections.

Line-of-sight post-processing operates after search: using Bresenham's line algorithm, it inspects consecutive waypoints and checks whether a direct straight-line connection is collision-free against the costmap (threshold > 150). If so, all intermediate waypoints are removed. For example, a 32-waypoint grid path is reduced to just 3 waypoints by replacing long grid-aligned sequences with direct connections.

Our results confirm this complementarity: the multi-objective cost function produces a path that already favours smooth directions, and the post-processing then aggressively removes the redundant grid waypoints. Without the cost function, the post-processing would start from a jagged standard A* path with sharp 90-degree turns, limiting how many waypoints can be safely removed (a straight line between two points separated by a sharp turn is more likely to collide with obstacles). Together, they reduced path lengths from 22-35 waypoints to just 2-3 per segment, which directly translates into faster execution times on 3 out of 4 segments.

3.6 Explain the difference between the path smoothing introduced in your global planning algorithm (enhanced A* with path smoothing) and the smoothing performed by the local controller (e.g., DWA). Why is it important to have both, and how do their objectives differ in terms of trajectory execution for a TurtleBot3?

The global planner and the local controller operate at different levels of the navigation stack and serve distinct purposes:

Global planner (our enhanced A*) plans a complete path from start to goal on the static costmap before the robot moves. Its multi-objective cost function penalises sharp turns and minimises travel time, while the line-of-sight post-processing reduces the path to a minimal set of waypoints. It operates on the full map, considers the overall route, but has no knowledge of the robot's real-time dynamics or unexpected obstacles.

Local controller (DWA) operates in real-time during execution. It samples candidate velocity commands (linear + angular), simulates short-term trajectories, and selects the one that best balances: staying close to the global path, avoiding dynamic/local obstacles, and respecting the robot's velocity and acceleration limits. It works on a small local costmap and replans at high frequency (~10-20 Hz).

Why both are important:

- Without global smoothing, the global path contains sharp 90-degree or 45-degree grid-aligned turns with many intermediate waypoints. DWA must then deviate significantly from the global path, which can cause oscillations or inefficient trajectories.

- Without DWA, the robot would blindly follow the global waypoints with no ability to react to localisation drift, dynamic obstacles, or kinematic constraints (e.g., TurtleBot3 cannot strafe laterally).

- A smoother global path with fewer waypoints gives DWA a better reference to track. This is confirmed by our results: with only 2-3 waypoints, the smoothed path lets DWA plan longer, more efficient local trajectories between widely-spaced waypoints, resulting in faster execution on most segments.

In summary, global smoothing optimises what path to follow, while DWA optimises how to follow it in real-time. Both are necessary for robust navigation on a differential-drive robot like TurtleBot3.

3.7 Were there any navigation failures or unstable behaviours (e.g. oscillations, frequent corrections) for either planner? If any were observed, how do these relate to the characteristics of the planned path and/or the environment?

Standard A*: We observed that the robot occasionally spun in place just before reaching a goal waypoint. This is a known behavior when the DWA local controller struggles to align the robot precisely with the final waypoint position, the robot overshoots or approaches from an angle that requires a rotation to match the goal orientation.

Enhanced A* with smoothing: We observed intermittent issues between segments 3 and 4, where the robot would occasionally get stuck. This behavior was non-deterministic, depending on the run, the robot would recover more or less quickly. The smoothed algorithm also produced movements where the robot would spin in place or reverse to re-align itself.

These issues can be attributed to the very low number of waypoints (2-3 per segment) produced by the line-of-sight post-processing. With so few waypoints spaced far apart, DWA has less guidance on the intended trajectory and must plan longer local paths autonomously. In complex areas of the maze, this can cause the local planner to temporarily lose track of the optimal trajectory, especially near obstacles where the straight-line global path passes close to walls.

The spinning behavior observed with both algorithms near goal waypoints relates to DWA's goal-reaching logic: when the robot is close to a waypoint but not precisely aligned, DWA prioritises rotation to match the goal pose, which can appear as the robot turning in circles before proceeding to the next segment.

These observations reinforce the finding from Q3 that the smoothed algorithm does not consistently improve execution despite its theoretical advantages and highlight the importance of compatibility between the global path and the local controller.